Graph Algorithms

Directions: Some of the questions on this assignment will appear on the quiz on Wednesday, October 20th. For

Design and analyze means:

- Give pseudocode for your algorithm.
- Prove that your algorithm is correct.
- Give the running time for your algorithm.

1. State the time complexity of the following operations if using either an adjacency matrix or adjacency list (with a linked list) as your graph representation, in terms of the number of vertices in the graph (n), the number of edges (m) and the number of neighbors of the queried vertex (deg(v))

    a. Check whether u and v are neighbors

    b. List all neighbors of v

2. State the space complexity of both the adjacency matrix and adjacency list representations

3. Extend the DFS-Explore(G,v) and DFS(G) algorithms so that, after running DFS(G) on an undirected graph, the following queries are supported:

    a. Identifying which vertices acted as root for the individual DFS-Explore calls

    b. Identifying who the parent of a given vertex is on the search tree

    c. Printing the path, along the search tree, from a given vertex to the root of its search tree

    d. Querying the pre- and post-visit number of each vertex

    e. Querying the type of any given edge (u,v)

    f. Given a back edge (u,v), print the corresponding loop

    g. Assigning a number to each new connected component, and querying for the component of a given vertex

4. Perform the DFS algorithm on the following graph. Give the pre- and post-order numbers for each vertex. Characterize each edge as a tree edge or back edge. For consistency, break ties alphabetically.You may alternatively request an adjacency list from the instructor

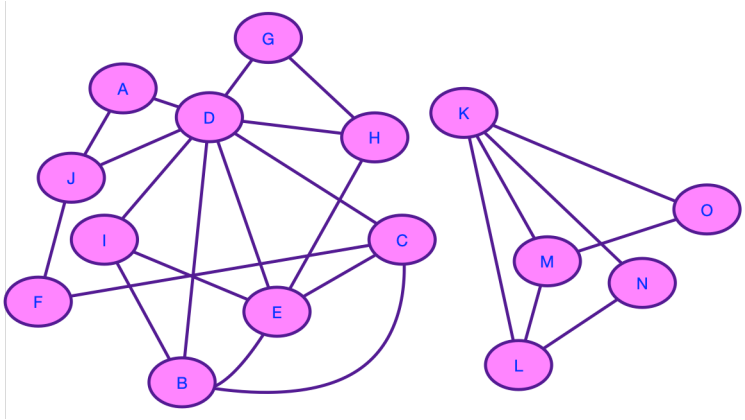


Figure 1 - Graph for question 1

5. Describe how to determine if a back edge in an undirected graph creates an odd length cycle using only pre and post order numbers for each vertex

6. A *bipartite graph* is a graph $G = (V, E)$ such that V can be partitioned into two sets ($V = V_1 \cup V_2$ and $V_1 \cap V_2 = \emptyset$) such that there are no edges between vertices in the same set. Equivalently: vertices can be colored "red" and "blue" such that no two vertices of the same color share an edge.

    a. Design and analyze an algorithm that determines whether an undirected graph is bipartite.

    b. Prove the following theorem: *An undirected graph, G, is bipartite if and only if it contains no cycles of odd length.*

7. There are three containers: a 10-liter container, a 7-liter container, and a 4-liter container. The 7- and 4-liter containers are initially full of water, while the 10-liter container is empty. There is only one operation: pour the contents of one container into the other stopping only when the pouring container is empty or the receiving container is full.

    a. Is there a sequence of pourings that leaves exactly 2 liters in the 4-liter container?

    b. Model this as a graph problem:

        i. Define the graph (What do vertices correspond to? What do edges correspond to?).

        ii. What is the problem that needs to be solved?

        iii. What algorithm can be used to solve this problem?

8. Suppose you had a tree with 14 vertices, A, B, ..., N. You wrote down the pre and post orderings for the vertices:Then you lost your graph.

    Preorder: A, B, D, E, H, L, M, I, F, C, G, J, K, N

    Post order: D, L, M, H, I, E, F, B, J, N, K, G, C, A

    Can you reconstruct the tree? If so, do.

9. Design and analyze an algorithm which takes as an input an undirected graph, G, and an edge, $e = (u, v)$, and determines whether G has a cycle containing e.

10. Prove that in an undirected graph, $\Sigma_{u \in V}\ \text{degree}(u) = 2|E|$. Conclude that there must be an even number of vertices with odd degree. Does a similar statement hold for the number of vertices of odd indegree in a directed graph?

11. Prove the following theorem:*In any connected undirected graph, there is a vertex whose removal leaves the graph connected*

12. Perform the DFS algorithm on the following directed graph. Assign pre- and post-visit numbers to each vertex and label each edge as a tree edge, forward edge, back edge or cross edge. For consistency, break ties alphabetically.

You may alternatively request an adjacency list from the instructor.

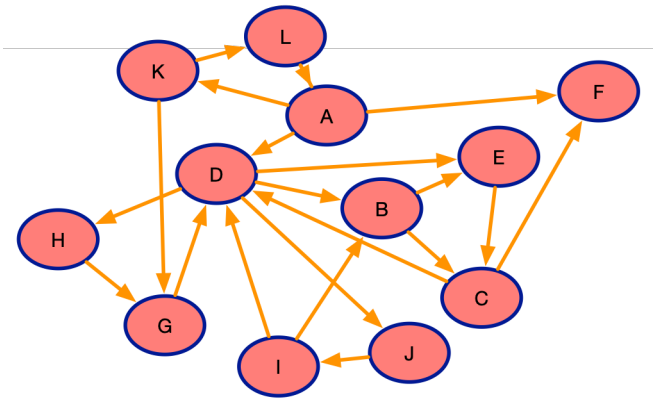


Figure 2 - Graph for problem 9

13. Design and analyze an algorithm for finding an odd length cycle in a strongly connected directed graph.

14. Is it always possible to make an undirected graph with two connected components connected by adding a single edge?

- Why or why not? (Proof or counterexample)
- Does the same hold true for directed graphs (and strongly connected components rather than connected components)? (Proof or counterexample)

15. Give an example of a strongly connected directed graph $G = (V, E)$ such that, for every $v \in V$, removing v from G leaves a directed graph that is not strongly connected.
16. True or False? (Proof or counterexample) If a depth-first search on a directed graph $G = (V, E)$ produces exactly one back edge, then it is possible to choose an edge $e \in E$ such that the graph without e ($G' = (V, E \setminus \{e\})$) is acyclic.
17. True or False? (Proof or counterexample) If a directed graph $G = (V, E)$ is cyclic but can be made acyclic by removing one edge, then a depth-first search in G will encounter exactly one back edge.
18. Modify the Breadth First Search algorithm to handle directed graphs.
19. Suppose a CS curriculum consists of n courses, all of them mandatory. The prerequisite graph G has a vertex for each course, and an edge from course v to course w if and only if v is a prerequisite for w. Design and analyze an algorithm that works directly with this graph representation, and computes the minimum number of quarters necessary to complete the curriculum (assume that a student can take any number of courses in one quarter).

20. Suppose you are given a source word and a target word. Your goal is to transform the source word to the target word in as few valid steps as possible. The valid steps are:

- change one character of the word to a new character
- add one character to the word
- remove one character from the word

After each step, the resulting string must be a valid English word. For example, we could transform *cat* to *hut* in two steps:

cat → *hat* → *hut*

Model this problem as a graph. What algorithm can be used to solve this problem?

21. Run Dijkstra's algorithm on the following graph, starting at vertex A.

You may alternatively request an adjacency list from the instructor.

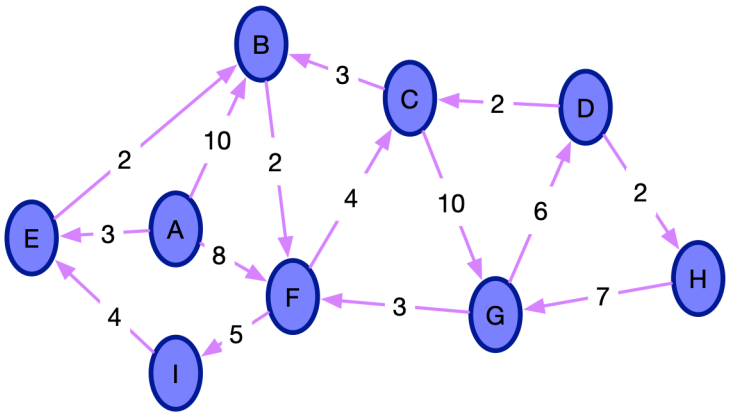


Figure 3 - Graph for problem 18

(a) Give a table showing the intermediate distance values for all vertices at each iteration of the algorithm.

(b) Give the final shortest path tree.

22. Can Dijkstra's algorithm handle negative weight edges? Why or why not?